

Physical AI SDK - Model Lifecycle & Strategy 2026

Hari Krishna Vydana
PAVS AI

Model Lifecycle Overview



Key Phases:

1. Enablement

- Get the model running
- Off-the-shelf installation (solve dependencies),
- benchmarking (respective metrics w.r.t.model);
- Identify the right dataset,
- Fill the docs(<http://localhost:8000>) with the initial results and expectation

2. Benchmarking

- Measure baseline performance
- Performance metrics
- Competitive analysis, With a right dataset on gpu vs cpu & gpu-npu
- Give inputs to CI/CD to automate it

Progressive Model Support Strategy

From Individual Models to System Orchestration

Phase 1: Individual Model Support

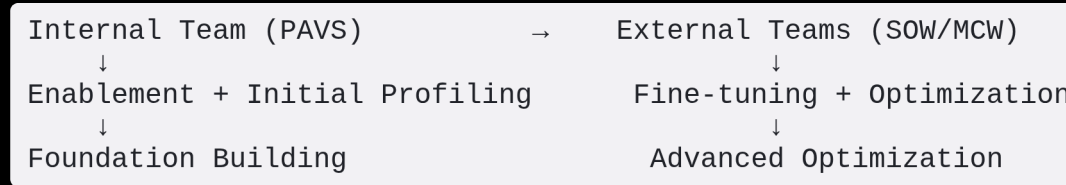
- Focus on single model enablement and optimization
- Establish baseline performance for each model
- Build reusable infrastructure and tooling
- Might get simple as we gain expertise

Phase 2: Multi-Model Orchestration

- Systems: Orchestration of multiple models working together (More complex)
- System-level optimization and resource management
- End-to-end pipeline integration
- Will give a real feel of product for every developer

Effective Team Collaboration

Progressive Engagement Model



Strategy:

- **PAVS Team:** Handle enablement and initial profiling
 - Establish working baseline
 - Identify optimization opportunities
 - Create clear specifications for external teams
- **External Partners:** Focus on fine-tuning and advanced optimization
 - Domain-specific optimization
 - Large-scale benchmarking
 - Production deployment preparation

Benefits:

- ✓ Better ROI on external resources
 - ✓ Clear handoff criteria
- ✓ Faster time to production

Speed Matters - Parallel Execution

Model Status Across Lifecycle Stages

Model Name	Category	Enablement	Profiling	Benchmarking	Fine-tuning
MobileSAM	Vision/Segmentation	PAVS	✓	AIG	🔄
YoloV12	Object Detection	✓	✓	✓	
Qwen3-VL	VLM	✓	✓	🔄	
LLaMA3.2-Vision	VLM	✓	✓		
Whisper	ASR	✓	✓	✓	✓
XTTS	TTS	✓	✓	✓	
DeepSeek-R1	LLM	✓	🔄		
Qwen2.5-VL	VLM	✓	✓	✓	🔄
CrossFormer	Vision Transformer	✓	🔄		
CenterPoint	3D Detection	🔄			
OpenVLA	Vision-Language-Action	🔄			
MedSigLIP	Medical Classification	🔄			
EfficientNetV2	Image Classification				
RAFT Stereo	Stereo Matching				

Legend: ✓ Complete | 🔄 In Progress | || Not Started

Key Insight: Start early, execute in parallel across multiple models

Slide 6: Environment & CI/CD Infrastructure

Isolated Environments for Each Model/System

Environment Strategy:

```
physical-ai-sdk/  
├── models/  
│   ├── mobilesam/  
│   │   ├── gpu/  
│   │   │   ├── Dockerfile  
│   │   │   ├── pyproject.toml (uv)  
│   │   │   └── .venv/  
│   │   └── npu_gpu/  
│   │       ├── Dockerfile  
│   │       ├── pyproject.toml (uv)  
│   │       └── .venv/  
│   └── yolov12/  
│       ├── gpu/  
│       │   ├── Dockerfile  
│       │   ├── requirements.txt (pip)  
│       │   └── .venv/  
│       └── npu_gpu/  
│           ├── Dockerfile  
│           ├── requirements.txt (pip)  
│           └── .venv/  
└── llama3-vision/  
    ├── gpu/  
    │   ├── Dockerfile  
    │   ├── pyproject.toml (uv)  
    │   └── .venv/  
    └── npu_gpu/  
        ├── Dockerfile  
        ├── pyproject.toml (uv)  
        └── .venv/
```

Benefits:

Slide 7: Container & Virtual Environment Options

Deployment Flexibility

Option 1: Dockerfile (Containerized)

```
FROM rocm/pytorch:latest
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "inference.py"]
```

Option 2: uv + pyproject.toml (Modern Python)

```
[project]
name = "mobilesam-inference"
version = "1.0.0"
dependencies = [
    "torch>=2.0.0",
    "onnxruntime-rocm>=1.17.0",
]
```

Option 3: pip + requirements.txt (Traditional)

```
torch==2.0.0
onnxruntime-rocm==1.17.0
opencv-python==4.8.0
```

Model Distribution

Optimized Models Repository:

Expanding Model Portfolio

Staying Current with Latest Open Source Models

Emerging Neural Alternatives:

-  **End-to-End Neural Pipelines** replacing traditional CV pipelines
 - Example: Neural SLAM replacing traditional SLAM
 - Example: Neural 3D reconstruction replacing classical methods
-  **Latest Model Categories to Track:**
 - Multimodal Foundation Models (GPT-4V alternatives)
 - Action-VLMs (RT-2, OpenVLA successors)
 - Neural Rendering (NeRF, Gaussian Splatting for robotics)
 - On-device Edge LLMs (< 3B parameters)

Strategy:

Slide 9: Benchmarking & Documentation Strategy

GitHub Pages Integration

Documentation as Code:

```
docs/
├── index.md                # Landing page
├── models/
│   ├── mobilesam.md       # Model-specific docs
│   ├── yolov12.md
│   └── benchmarks/
│       ├── robotics.md    # Benchmark results
│       └── industrial.md
├── guides/
│   ├── quick-start.md
│   └── deployment.md
└── _config.yml            # GitHub Pages config
```

Automated Workflow:

1. Code + Benchmarks → GitHub Repo

Slide 10: GitHub Pages Example - Mock Preview

Sample Documentation Site Layout

Homepage Preview:

Physical AI SDK Documentation

Quick Start | Benchmarks | Models

Model Catalog

- Vision Models (12)
 - MobileSAM, YoloV12, EfficientNet...
- VLM Models (5)
 - Qwen3-VL, LLaMA3.2-Vision...
- Audio Models (3)
 - Whisper, XTTS...

Performance Dashboard

Model	FPS	Latency	Acc
MobileSAM	240fps	4.2ms	94%
YoloV12	240fps	4.2ms	92%
Whisper	100tok/s	10ms	96%

Slide 12: Practical Example - Model Lifecycle

Case Study: Qwen3-VL Integration

Enablement:

-  Model weights download and conversion
-  Basic ONNX-RT integration
-  Simple inference test (single image)
-  Docker environment setup

Profiling

-  Layer-wise performance analysis (AMD Profiler)
-  Memory usage tracking
-  Hotspot identification (Conv3D, GEMM operations)

Week 5-6: Benchmarking

-  Compute the relevant performance on a dataset wrt to a leaderboard or paper
-  KPI testing: Target 30fps @ 1080p
-  Accuracy validation on VQA datasets
-  Power consumption measurement

Fine-tuning:

-  kernel optimization for GEMM
-  INT8 quantization (W8A8)

Resource Management Strategy

Balancing Internal & External Resources

Resource Allocation Matrix:

Lifecycle Stage	Internal Team (PAVS)	External Partners	Duration
Enablement	■■■■■■■■■■ (Primary)	■■■ (Support)	2-3 weeks
Profiling	■■■■■■■■ (Lead)	■■■■ (Assist)	2-4 weeks
Benchmarking	■■■■■ (Guide)	■■■■■■■ (Execute)	3-4 weeks
Fine-tuning	■■■ (Review)	■■■■■■■■■■ (Primary)	4-8 weeks

Handoff Criteria:

- ✓ Model runs successfully on target hardware
 - ✓ Profiling report completed
 - ✓ Baseline benchmarks established
- ✓ Optimization opportunities documented

Slide 14: Technology Watchlist 2026

Emerging Models & Technologies

High Priority:

1. **Qwen3-VL 32B** - Latest multimodal foundation model
2. **RT-X** - Robotics transformer with broader action space
3. **Neural SLAM** - End-to-end learned SLAM systems
4. **Depth Anything V2** - Improved monocular depth estimation

Medium Priority:

5. **SAM 2** - Segment Anything Model v2 with video support
6. **Florence-2** - Microsoft's vision foundation model
7. **Stable Diffusion 3** - For synthetic data generation
8. **Phi-3 Vision** - Compact VLM (< 4B params)

Success Metrics & KPIs

Measuring Lifecycle Effectiveness

Model Lifecycle KPIs:

Metric	Target	Current	Trend
Average Time to Enablement	< 2 weeks	1 weeks	↓ Improving
Models in Profiling Stage	5-8 concurrent	6	→ Stable
Benchmark Pass Rate (1st try)	> 80%	75%	↑ Improving
Fine-tuning Speedup (avg)	> 2x	2.3x	↑ Good
CI/CD Pipeline Success Rate	> 95%	97%	✓ Excellent

Portfolio Metrics:

- Total Models Supported: **45+**
- Production-Ready Models: **28**
- Models in Development: **12**
- Experimental/Research: **5**

Roadmap & Next Steps

2026 Execution Plan

Q1 2026 (Current):

-  Establish lifecycle framework
-  Complete 8 models through full lifecycle
-  Deploy GitHub Pages documentation

Q2 2026:

-  Expand to 15 production-ready models
-  Implement automated benchmarking CI/CD
-  Onboard 2 external partner teams (MCW/SOW)

Q3 2026:

-  Multi-model orchestration (5 system pipelines)
-  Add 10 emerging models from watchlist